

Developing Edge AI Applications for Human-Robot Collaboration in LEGO Assembling Operations

Kayla Hicks¹, Toluwani Olasoko², Abdulahi Oyeibanji³, Joaquin Snowden⁴, Derrick Mirindi⁵, Yuhan Jiang^{6,7,*}

¹Cybersecurity Intelligence Management, Morgan State University, Baltimore, USA, kahic10@morgan.edu

²Electrical Engineering and Computer Science, Howard University, Washington, DC, USA, toluwani.olasoko@bison.howard.edu

³School of Computing Sciences and Computer Engineering, University Of Southern Mississippi, Hattiesburg, USA, abdulahi.oyebanji@u

⁴School of Engineering, Morgan State University, Baltimore, USA, joaquinsnowden27@gmail.com

⁵School of Architecture and Planning, Morgan State University, Baltimore, USA, demir1@morgan.edu

⁶Center for Equitable Artificial Intelligence and Machine Learning Systems, Morgan State University, Baltimore, USA, yuhan.jiang@mor

⁷Department of Civil and Environmental Engineering, Morgan State University, Baltimore, USA, yuhan.jiang@morgan.edu

Abstract—This study evaluates and optimizes YOLO-based object detection for recognizing and localizing small, visually similar LEGO pieces in a real-time human-robot assembly setting. We trained twenty-five (25) YOLO models on the B200 LEGO detection dataset, spanning the YOLOv8, YOLOv8-p2, YOLOv11, YOLO26, and YOLO26-p2 families across the nano, small, medium, large, and extra-large scales. Each model was compared on precision, recall, F1 score, IoU, and mAP across training, validation, and testing. From this benchmark, we selected YOLOv11s for real-time detection: it offers the best accuracy-latency trade-off for driving our robotic arm, enabling reliable, automated pick-and-place LEGO assembly. ...

Index Terms—YOLO, Nvidia Jetson, LEGO, Robot

I. INTRODUCTION

Artificial intelligence (AI), deep learning (DL), and robotic technologies have dramatically improved autonomous systems in industrial, agricultural, service, and home environments. Modern robots no longer must execute pre-programmed movements. Instead, they rely on computer vision to perceive their environment, recognize objects, make decisions, and complete complex physical tasks. You Only Look Once (YOLO) is one of the most popular real-time computer vision techniques for robotic perception. It was developed to detect and localize objects at the same time, in a single neural network architecture [1]–[3].

Understanding the environment is essential for robotic intelligence, and computer vision is a fundamental part of that. Traditional machine vision approaches often struggle with varying lighting conditions, object occlusion, noisy environments, and real-time processing demands. DL-based object detection models, especially YOLO variants, have driven major improvements in detection accuracy and computational efficiency [3], [4]. These gains have made YOLO systems increasingly common in fields that need rapid, accurate object identification, such as robotic grasping [5], industrial automation [4], self-navigation, agricultural harvesting, and assistive robotics.

This work was supported by the Center for Equitable Artificial Intelligence and Machine Learning Systems (CEAMLS) at Morgan State University.

YOLO-based vision systems have been shown to generalize across many fields of robotics. In industrial settings, YOLO has been used to guide robotic arms for intelligent sorting. One system paired YOLOv5-based visual detection with a six-axis robotic arm and a coordinate transformation model linking the camera view to arm movement. This enabled real-time bottle cap recognition, spatial positioning, and automated grasping for sorting in the food, chemical, and pharmaceutical industries [3]. YOLO has also been applied to robotic grasping using oriented bounding boxes, which localize an object's position, orientation, and class simultaneously. This enables accurate pick-and-place operations in the real world [5]. Together, these studies show that YOLO can extend beyond simple object recognition to become a core part of the robotic manipulation pipeline.

YOLO-based perception has also significantly advanced agricultural robotics. YOLO architecture has been used for tomato harvesting [6], grape detection [7], strawberry growth monitoring [8], and apple detection [9], [10]. These systems can recognize the target crop, predict picking points, and guide the robotic manipulator in complex field environments and different lighting conditions. The baseline YOLO architecture has been further improved through attention mechanisms, feature fusion, and lightweight network designs. These improvements help the models stay accurate while remaining suitable for embedded robotic platforms [9]–[11]. This body of work shows how YOLO-based models enable real-time decision-making and action in dynamic environments.

YOLO has also proven to be effective in assistive and service robotics, beyond industrial and agricultural use. A YOLOv8-based system (using the n, s, m, and l variants) has been developed to classify kitchen utensils, supporting kitchen safety and accessibility for users with visual or physical impairments [12]. A separate YOLO-based human detection robot was built on a resource-constrained embedded microprocessor. It achieved near real-time image processing, at 0.5–1 second per image, with 75–80% detection accuracy. This speed is too slow for applications requiring instantaneous reaction, such as dynamic autonomous navigation. However, it proved

sufficient for the security and surveillance context it was designed for [13]. This shows that YOLO-based perception can run effectively on low-power hardware if response-time requirements are matched to the application. Together, these studies show that YOLO can be tailored for tasks such as defect detection [11], pose estimation in humanoid robots [14] and toy identification and categorization for home-organization systems [15]. Together, these studies show that YOLO models can perform well across many operational contexts, though the right balance of speed and accuracy still depends on the demands of the specific task.

Despite these advances, challenges remain around small object manipulation. Such tasks require precise recognition, localization, orientation estimation, and sequential assembly. Most of the existing work focuses on object detection or sorting, not complex assembly operations that require understanding relationships among multiple components [12], [15]. Industrial and agricultural applications mostly call for repetitive pick-and-place actions. Assembly tasks, by contrast, demand greater accuracy, differentiation, and coordination between perception and manipulation systems. Several studies also note ongoing challenges with occlusion, small object detection, environmental variation, and generalization to real-world conditions [2], [6], [10], [14].

Despite significant advances in YOLO-based object detection for robotics, most existing studies focus primarily on object recognition, sorting, and simple pick-and-place operations in structured environments such as manufacturing lines or agricultural fields. Although these applications demonstrate strong detection accuracy and real-time performance, they rarely address complex robotic assembly tasks that require sequential reasoning, precise spatial awareness, and interaction among multiple small components. Existing systems often prioritize detecting isolated objects rather than understanding inter-object relationships, orientation dependencies, and assembly order, all of which are critical in automated construction tasks.

The LEGO assembly process presents a particularly challenging benchmark for robotic vision and manipulation systems. LEGO components are typically small, visually similar, and available in many colors, shapes, and dimensions. Their similarity increases the difficulty of classification, especially under varying lighting conditions, partial occlusion, or cluttered arrangements. Furthermore, successful robotic assembly requires more than recognizing individual pieces. The system must accurately determine the type, orientation, position, and functional role of each component within a predefined assembly sequence. Even minor detection or localization errors may cause incorrect grasping or assembly failure.

A major research gap therefore exists in applying YOLO-based computer vision models to fine-grained robotic assembly involving small, highly similar objects such as LEGO bricks. Limited research has investigated how modern YOLO architectures can be integrated with robotic manipulation systems to support reliable end-to-end assembly workflows. This study aims to address this gap by evaluating and optimizing YOLO-based object detection models for LEGO piece recognition and

localization in a robotic assembly environment. Specifically, the research seeks to identify the model configuration that provides the best balance between detection accuracy, inference speed, and robustness for real-time robotic manipulation and automated LEGO assembly.

II. BACKGROUND

In everyday livelihood, self-driving cars and security cameras can instantly recognize people, vehicles, and other objects by using artificial intelligence models that can process visual information in real time. One of these models is YOLO, which stands for “You Only Look Once”. Introduced in 2015, YOLO revolutionized computer vision. Unlike many earlier object detection methods that process an image in multiple stages, YOLO analyzes the entire image in a single forward pass, which makes it perform real-time object detection. In addition, newer YOLO versions are easier to train and have the capacity to be more efficient in both speed and computational cost, making them suitable for a wide range of real-world applications.

Before discussing the development of YOLO (You Only Look Once), it is important to understand the fundamental issue of object detection that lies at the heart of computer vision. Object detection includes recognition of an object class in an image as well as localization of the object within a bounding box or other localization technique. Object detection is thus more complex than simple image classification, since it can simultaneously answer what objects are present and where they are located.

Recent developments in DL technologies have dramatically changed the field of object detection, enabling models to learn hierarchical visual features directly from vast amounts of data without relying on manually engineered features. This transition has enhanced the robustness against challenging conditions such as occlusion, variable illumination, cluttered scenes, and dynamic environments. In the case of robotic systems, these improvements are crucial, as the robot needs to accurately perceive objects in a timely manner to enable real-time decision making and physical interaction with the environment [1], [2], [4].

YOLO solved a significant problem of paradigm shift: The problem of detecting objects could be transformed into a single-stage regression problem. In 2016, a YOLO model was proposed by Joseph Redmon and colleagues that breaks away from the two-stage pipeline of region proposal and classification by training a single forward pass through a neural network for object localization and classification [20]. This is the architecture that splits an image into a grid and predicts the bounding boxes, confidence scores, and class probabilities simultaneously in each grid cell. Consequently, YOLO became the fastest detector, enabling real-time computer vision applications in robotics and embedded systems [20], [22].

However, YOLOv1 has some drawbacks due to its speed. The accuracy of the localization was poor, especially for small objects or many objects close together in the same localization cell. This led to inaccurate or failed bounding boxes in

dense scenes. Its weaknesses hindered it from performing fine-grained robotic manipulation tasks, where localization is crucial, such as grasping and assembly tasks involving small parts [13], [20].

VGG stands for Visual Geometry Group, the research group at the University of Oxford that created the model. In order to overcome these limitations, YOLOv2, inspired by the VGG architecture, was released in 2016, which makes several improvements to the architecture, such as using batch normalization, anchor boxes, multi-scale training, and processing input data in higher resolutions [21]. These changes resulted in better recall and localization accuracy, and the ability to detect in a much wider range of categories. The addition of anchor boxes greatly enhanced the accuracy of the bounding box prediction, particularly in the case of objects with different aspect ratios and scales. This enhancement gave YOLO more strength to be used for implementing it in the real environment with complex conditions [21], [22]. Around the time this was released, YOLO9000 was also released. YOLO9000 can detect about 9000 different object categories.

In 2018, many of those strides were made with YOLOv3, the multi-scale prediction through the feature pyramid and residual connection via Darknet-53. It was introduced in a technical paper by its original creators Joseph Redmon and Ali Farhadi. One of the key strengths of the earlier version of YOLO [22] was its failure to detect small, medium, and large objects simultaneously. Multi-scale detection was able to solve this by improving the model's performance on all three sizes of objects. This improvement is especially important in robotic LEGO assembly, where LEGO bricks are often in small areas within the image, and may be partially occluded or grouped together with similar-looking LEGO parts. Robust performance in small-object feature extraction and localization is necessary for accurate detection of such objects. [13], [15].

Released in 2020, YOLOv4, developed by Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao, bridged a critical gap in the notion of computer vision. YOLOv4 improved the Darknet framework. These improvements allowed YOLOv4 to achieve higher accuracy while maintaining real-time detection speeds, the YOLO family was further developed and expanded by YOLOv4, which added Cross Stage Partial (CSP) networks, mosaic data augmentation, and multiple optimization strategies for maintaining real-time speed and boosting detection accuracy [23]. This led to the model being very appealing to hardware-constrained embedded robotic platforms [23].

Later variants further enhanced the speed-accuracy compromise. YOLOv5 was adopted by many industries thanks to its efficient implementation and ease of deployment, and YOLOv7 added two trainable optimization strategies to attain state-of-the-art real-time object detection performance [24]. In recent years, the YOLOv8 and subsequent iterations (including YOLOv11 and YOLOv26) have integrated anchor-free detection heads and enhanced backbone network architectures and feature fusion methods. These improvements are able to improve the generalization ability, inference speed, and small

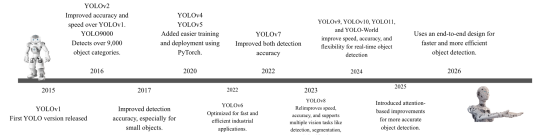


Fig. 1. YOLO Timeline

object detection ability, which is very suitable for fine-grained applications in robots [17], [18].

YOLO architectures have become an integral part of robotic perception systems in various domains in recent years. In industrial robotics, YOLO-based systems have facilitated high spatial precision sorting and grasping by a robot in real-time. In fact, the YOLO-guided robotic arms have been successfully used in sorting bottle caps [3] and intelligent manipulation of objects in manufacturing environments [5]. YOLO-based detection has successfully facilitated automatic tomato harvesting in complex environments, as well as strawberry, grape, apple, and pepper harvesting [6]–[11], all of which are related to agricultural robotics. The examples show how YOLO architectures can be adapted to different operational scenarios.

YOLO has also demonstrated great abilities in the field of assistive and service robotics. They are applied in kitchen safety, in which the utensils are classified [12]–[14] and human detection for a surveillance robot [15], and in distance estimation for humanoid robots [12]–[14]. The results highlight the fact that current YOLO models are capable of accurate perception in resource-limited embedded hardware with an appropriate trade-off between computing cost and detection accuracy.

While YOLO has succeeded in detecting and robotic perception, there are still many challenges in fine-grained assembly tasks. In contrast to the aforementioned existing YOLO-based robotic systems, which are used for object detection or pick-and-place operations, most of the existing systems are not intended for sequential assembly tasks of multiple visually similar components. LEGO robotic assembly is an impressively difficult benchmark due to the need to recognize, locate, and estimate the orientation of small objects as well as manipulate them in a sequence-aware manner. Thus, assessing modern YOLO variants for these tasks can be used to gain insight into the applicability of computer vision advancements to enhance autonomous robotic assembly systems.

III. YOLO PERFORMANCE METRICS

Performance metrics are measurements used to evaluate how well a ML model performs. In YOLO-based object detection, these metrics help compare models beyond visual inspection by showing how accurately the model detects, classifies, and locates objects (see Table 1). Erickson and Kitamura explain that classification metrics can be grouped into binary, multiclass, and multi-label tasks, and that common evaluation outcomes include true positives, false positives, true

negatives, and false negatives [25]. For this project, these metrics are important because the robot must correctly identify LEGO pieces before performing pick-and-place operations. A true positive occurs when the model correctly detects the target LEGO piece. A false positive occurs when the model incorrectly labels another object as the target. A false negative occurs when the model does not detect a LEGO piece that is present. A true negative occurs when the model correctly ignores objects that are not the target. These values are used to calculate precision, recall, and the F1 score. Precision, also called positive predictive value, measures the fraction of predicted positives that are correct. Recall, also called sensitivity, measures the fraction of actual positives that the model successfully detects. The F1 score is the harmonic mean of precision and recall, showing the balance between correct detections and missed objects. In YOLO evaluation, Intersection over Union (IoU) measures how much the predicted bounding box overlaps with the ground-truth box. Mean Average Precision (mAP) then summarizes the model’s overall detection performance by averaging Average Precision scores across all LEGO classes.

TABLE I
OBJECT DETECTION EVALUATION METRICS

Metric	Equation	Why It Matters
Intersection over Union (IoU)	Area of Overlap / Area of Union	Determines other metrics
Precision (P)	$TP / (TP + FP)$	Higher the precision the less false alarms
Recall (R)	$TP / (TP + FN)$	Higher the recall the less missed items
F1 Score	$2 \times (P \times R) / (P + R)$	Shows balance between P and R
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall correctness of model
Average Precision	$AP = \sum_n (R_n - R_{n-1}) P_n$	Evaluates the quality of classes
mAP@50	Mean of AP across classes	Average precision across the full model

TP = True Positive; FP = False Positive; TN = True Negative; FN = False Negative

IV. DATA AND METHODS

Our dataset came from B200 LEGO on Kaggle, which contained 2,000 LEGO bricks with XML annotations marking the boundary boxes and part numbers across 200 part types. This model is a supervised machine learning model, and the deep learning Computer Vision algorithm being used is YOLO. We chose YOLO because it can perform real-time object detection across multiple classes and still maintain its high accuracy across multiple different classes.

Before we started training, we converted the XML annotations into YOLO’s text label format. This lists the class index together with the normalized center coordinates, width, and height for each box. We then split the data set into 1400 training images, 300 validation images, and 300 test images, which is the standard 70/15/15 split for object detection work. The training images teach the model, the validation

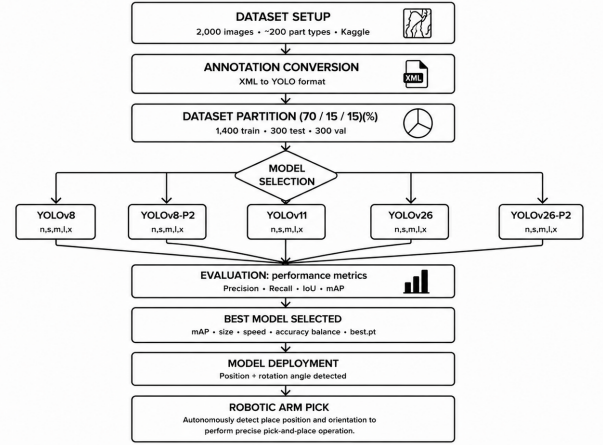


Fig. 2. Flowchart of the study

images check the progress mid-training, and the test remains untouched until the final evaluation of the model performance.

To study the model’s training behavior and result, we kept the data augmentation constant, varying only the number of epochs. We tracked the precision, recall, mAP, IoU, and observed that as the epochs increased, all four metrics gradually increased, before it flattened, and the validation performance occasionally dipped which shows a sign of overfitting. After that, we used a fixed number of epochs and data augmentation across all 25 models we were training.

The 25 trained models were YOLOv8, YOLOv8-p2, YOLOv11, YOLO26, and YOLO26-P2, each in nano(n), small(s), medium(m), large(l) and extra large(x) sizes. The model feeds directly into the LEGO Cobot’s robotic arm, which picks bricks based on detected position and rotation angle. A wrong detection means a wrong pick, so model choice depends on both accuracy and speed on the Jetson hardware that runs it.

V. RESULTS

After training the 25 YOLO model variants, we analyzed the results.csv file generated after each model training. We then compared the models using various performance metrics such as the precision, recall, mAP@50, mAP@50-95, training time, F1-score, and validation loss. Comparing across various metrics made us evaluate the model performance as well as the time and suitability for deployment on our Edge AI computer.

An unexpected result was that the p2-architecture variants did not perform as well as the other models. P2 models are expected to perform better for small-object detection. However, our results showed that strong performance depends more than on object size alone. Factors such as dataset quality, image scale, class similarity, and training conditions also affected model performance. Of the 18 models shown in the evaluation table, the P2 variants ranked near the bottom. For example, YOLOv8P2x achieved only 19.43% precision and 20.38% average recall, which was substantially lower than

TOP 18 MODELS

Rank	Model	mAP50-95	mAP50	Precision	Recall	Train (min)	Loss
1	yolo11l	0.7012	0.7367	0.9638	0.7403	607.6	0.292
2	yolo11x	0.6967	0.7238	0.9670	0.7329	730.6	0.268
3	yolo11m	0.6931	0.7330	0.9536	0.7370	500.5	0.308
4	yolov8x	0.6856	0.7153	0.9523	0.7267	626.2	0.275
5	yolo26l	0.6810	0.7202	0.9444	0.7259	792.7	0.350
6	yolov8l	0.6746	0.7093	0.9442	0.7215	584.9	0.291
7	yolo26m	0.6691	0.7111	0.9372	0.7188	720.5	0.370
8	yolo11s	0.5946	0.6432	0.8685	0.6658	376.5	0.390
9	yolo26s	0.5378	0.5828	0.7984	0.6203	525.1	0.472
10	yolov8s	0.4834	0.5309	0.7445	0.5766	341.4	0.423
11	yolo11n	0.3165	0.3504	0.5151	0.4347	333.2	0.559
12	yolo26n	0.2752	0.3101	0.4778	0.4029	454.7	0.692
13	yolov8n	0.2727	0.3049	0.4856	0.3942	303.8	0.548
14	yolov8p2n	0.2142	0.2472	0.3987	0.3423	813.7	0.673
15	yolo8p2m	0.1452	0.1664	0.2875	0.2650	1034.2	0.670
16	yolov8p2s	0.1234	0.1443	0.2415	0.2418	885.6	0.766
17	yolov8p2x	0.0902	0.1098	0.1943	0.2038	820.9	0.945
18	yolo26p2n	0.0655	0.0810	0.1445	0.1656	1538.9	1.092

Fig. 3. The Top 18 models

expected and showed weak detection performance for this dataset.

Among all the models we evaluated, YOLO11 demonstrated the best performance overall. Across all YOLO11 variants (nano, small, medium, large, and extra-large), the models achieved an average precision of 85.36% and an average recall of 66.21%, this was better than other models in accuracy, as well as robustness. In particular, YOLO11l had the highest mAP@50-95 score of 0.7012 which made it the best performing model in terms of the object detection accuracy.

Although larger YOLO11 variants had better accuracy and better performance, this was not the sole criteria for selecting our model. Since we were going to deploy the model on our EDGE AI computer, factors like speed, memory usage and real-time were also considered. Larger models usually require much more computational power, and time and this was not the optimal choice for real-time robotic applications.

As a result, YOLO11s was selected as the model to be deployed. Although the model did not offer the best result in overall accuracy, it offered the best trade-off between detection performance and computational efficiency. Smaller variants require less computational power and offers faster object detection which made it suitable for the real-time application on the Edge AI device.

After selecting YOLO11s, we further refined the model by selecting the top 30 LEGO classes that had the highest frequency of appearing in the training data. Reducing the classes reduced the model complexity and helped improved the model performance in real-world detection.

The final YOLO11s was then deployed on the Edge AI computer where it provided real-time object detection while operating within the hardware constraints of the embedded system. The model selection and refinement process ensured we had a balance between model performance, computational efficiency, as well as the practicality in using the model in real-time LEGO assembling operations.

VI. DISCUSSION

This study investigates whether computer vision and edge AI achieve reliable real-time detection and assembly of LEGO bricks with a robotic arm. As of the results, they sufficiently support our hypothesis that integrating computer vision and edge AI with a robotic arm enables accurate, real-time detection of LEGO bricks from a mixed pile.

For the experiment, the YOLO11 architecture outperformed both P2 architecture models. It emerged as the best fit for real-time LEGO detection and automating the robot arm. Additionally, YOLOv11s a recall of 95% training, indicating that the model was highly effective at identifying LEGO pieces during training. With this in consideration, the model also had the issue of over-detection, occasionally seeing objects that were not present, more commonly known as false positives.

Although our project's findings are consistent with prior research regarding the fact that YOLO-based object detection models are effective for real-time vision tasks, there are some irregularities. For instance, based on research, it was commonly assumed that the P2 architecture would improve the detection of objects. However, it didn't. For example, Li et al. (2021) improved YOLOv4-tiny for grape detection under varying lighting conditions and achieved a precision of 92.21%. Our research extends the concepts in this study by applying modern YOLO architectures to LEGO object detection and integrating the detection results with robotic hardware for automated assembly.

Our study focused on specific LEGO datasets and hardware. We believe the overall approach can generalize to other environments. This includes industrial automation and smart manufacturing. More specifically,

Limitations were mostly environmental based, including lighting and shadows. In addition, limitations in robotic arm calibration affected the consistency of successful pick-and-place operations. Because we were on resource constrained hardware, GPU memory limited the number of epochs and model configurations that could be tested.

As for future work, it should be focused on improving robotic grasping accuracy, updating robot code to place legos & apply pressure, and training more YOLO models.

In conclusion, combining computer vision and edge AI with a robotic arm can successfully enable reliable real-time detection of our LEGO bricks. The experiment showed that efficient computer vision can support automated robotic tasks without the reliance on cloud computing.

VII. LIMITATIONS

During YOLO model training and evaluation, we developed a chart that would present the top 30 classes across each model that would detect out of the 200 classes. However, the stumbling block was that more than 85% of those top performing classes were not available for us to use for our robotic arm test. Therefore, we had to adjust our testing approach in which we would select the common LEGO pieces such as 1x1, 2x2, 2x4, etc. This became a limitation because the physical robot testing did not fully represent

the complete 200-class dataset or the top-performing classes identified during model evaluation. Although the selective subset allowed us to continue developing and testing the detection-to-robot pipeline, it limited the ability to generalize the robotic pick-and-place results across all LEGO part classes. Another limitation during our evaluation was the model's difficulty distinguishing between LEGO pieces that had the same number of rows and columns but differed in height. This would happen when the pieces were viewed from the top standing upright because the camera could only see the length and width, not the height. As a result, pieces with similar top-view dimensions could be misclassified, even if they were different LEGO parts. In some cases, the team had to flip the pieces to the side or adjust their orientation so that the height difference became more visible to the camera, which then again results in a more biased test because the main goal is to allow our robot to detect and pick up any piece correctly, even if it is similar or occluded. This is very important to us because LEGO assembly requires LEGO assembly. Our robot to learn the distinction between every type of piece.

VIII. FUTURE WORK

Future work will focus on improving the YOLO training dataset by adding images from multiple angles, side views, tilted views, and different object orientations. This would eliminate our limitation because now we can better distinguish between LEGO pieces that have similar row-and-column layouts but different heights. Another future improvement is updating the robot control code so that the robotic arm can apply controlled pressure when attaching one LEGO piece to another, since assembly requires more than just picking and placing. Future work will also explore LEGO assembly simulation, allowing the team to test movement paths, placement accuracy, and possible collisions before using the physical robot. In general, these improvements would help move the system from pick-and-drop to a more complete robotic assembly pipeline.

IX. CONCLUSION

This project studied how YOLO object detection can support robotic LEGO assembly. The goal was to help a robotic arm identify, locate, pick, and move the correct LEGO piece from a pile. This is a challenging task because LEGO pieces are small and often look very similar. Some pieces only differ by color, length, height, or small shape details.

To complete this work, LEGO image data was prepared for YOLO training. The annotations were converted into YOLO text format, and the dataset was organized for model training and testing. In total, 25 YOLO models from five architecture families were trained and compared on the B100 LEGO dataset. The models were evaluated using key object detection metrics, including precision, recall, F1 score, Intersection over Union, and mean Average Precision. These metrics helped show how well each model detected the correct LEGO pieces, avoided false detections, and reduced missed objects.

The results showed that YOLO models can detect LEGO pieces with strong potential for robotic assembly. Among the tested models, YOLOv11s was selected as the most suitable model for real-time detection and robot-arm automation. Its performance was consistent across the training, validation, and test sets, which shows good generalization with minimal overfitting. YOLOv11s also provided a strong balance between accuracy and speed. This is important because, in live robotic pick-and-place operations, a wrong detection can lead to the robot picking the wrong LEGO piece.

Overall, this research shows that YOLO-based computer vision can support real-time LEGO detection and robotic pick-and-place tasks. The system has moved beyond model training by integrating detection with the robotic arm to pick LEGO pieces. Future work should focus on moving from pick-and-place toward full LEGO assembly. One key next step is adding LEGO building instructions to the robot so it can follow an assembly sequence and start building structures. Future work should also improve the training dataset by adding images from multiple angles, side views, tilted views, and different object orientations. These improvements would help move the system toward a more complete robotic LEGO assembly pipeline.

REFERENCES

- [1] V. C. Mahaadevan et al., "Integrated design and YOLO based control framework for autonomous EV charging robot platforms," *Results in Engineering*, 2025, doi: 10.1016/j.rineng.2025.105438.
- [2] J. Li, Y. Liu, S. Wang, L. Wang, Y. Sun, and X. Li, "Visual perception system design for rock breaking robot based on multi-sensor fusion," *Multimedia Tools and Applications*, vol. 83, no. 8, pp. 24795–24814, 2024, doi: 10.1007/s11042-023-16189-w.
- [3] Y. Wang, M. Xue, Y. Cai, Y. Gao, and P. Yan, "A Bottle Cap Intelligent Sorting System for Mechanical Arms Based on YOLOv5 Visual Guidance," in *2026 5th International Conference on Electronic Technology and Artificial Intelligence (ETAI)*, pp. 850–853, 2026, doi: 10.1109/ETAI68332.2026.11485387.
- [4] M. Makulavičius, J. J. Petronienė, E. Šutinys, V. Bučinskas, and A. Dzedzickis, "Industrial Robotic Setups: Tools and Technologies for Tracking and Analysis in Industrial Processes," *Applied Sciences*, vol. 15, no. 18, 2025, doi: 10.3390/app151810249.
- [5] C. D. Vo, L. H. Phuong, and L. D. Hanh, "Object detection using YOLO oriented bounding box for robot grasping applications," *International Journal of Cognitive Computing in Engineering*, vol. 7, no. 1, pp. 569–581, 2026, doi: 10.1016/j.ijce.2026.05.007.
- [6] Q. An, K. Wang, Z. Li, C. Song, X. Tang, and J. Song, "Real-Time Monitoring Method of Strawberry Fruit Growth State Based on YOLO Improved Model," *IEEE Access*, vol. 10, pp. 124363–124372, 2022, doi: 10.1109/ACCESS.2022.3220234.
- [7] G. Zhaoxin, L. Han, Z. Zhijiang, and P. Libo, "Design a Robot System for Tomato Picking Based on YOLO v5," *IFAC-PapersOnLine*, vol. 55, no. 3, pp. 166–171, 2022, doi: 10.1016/j.ifacol.2022.05.029.
- [8] H. Li, C. Li, G. Li, and L. Chen, "A real-time table grape detection method based on improved YOLOv4-tiny network in complex background," *Biosystems Engineering*, vol. 212, pp. 347–359, 2021, doi: 10.1016/j.biosystemseng.2021.11.011.
- [9] X. Du, X. Zhang, T. Li, X. Chen, X. Yu, and H. Wang, "YOLO-WAS: A Lightweight Apple Target Detection Method Based on Improved YOLO11," *Agriculture*, vol. 15, no. 14, pp. 1–19, 2025, doi: 10.3390/agriculture15141521.
- [10] Y. Huang et al., "Pepper-YOLO: a lightweight model for green pepper detection and picking point localization in complex environments," *Frontiers in Plant Science*, vol. 15, pp. 1–14, 2024, doi: 10.3389/fpls.2024.1508258.

- [11] H. Zhao, J. He, Y. Li, M. You, and L. Chen, "A lightweight YOLO-based model for accurate detection of red pepper clusters in robotic harvesting," *Scientific Reports*, vol. 16, no. 1, p. 5879, 2026, doi: 10.1038/s41598-026-36671-3.
- [12] N. S. Rakib, S. Karungaru, K. Terada, and M. M. Hasan, "Utensil Classification using Computer Vision: A Feasibility Study on Preventing Kitchen Accidents for Diverse Users," pp. 318–325, 2026, doi: 10.1109/ICIIBMS66230.2025.11316754.
- [13] R. Gholami, H. Dehdashti Jahromi, and S. Sedaghat, "Design and Implementation of a Near Real-Time Human Detection Robot Using YOLO Framework and IoT Technologies," *IEEE Access*, vol. 13, p. 175960, 2025.
- [14] S. Dwijayanti, B. Y. Suprpto, Mutiyara, and Rendyansyah, "Real-time object detection and distance measurement for humanoid robot using you only look once," *Bulletin of Electrical Engineering and Informatics*, vol. 13, no. 6, pp. 4134–4146, 2024, doi: 10.11591/eei.v13i6.7476.
- [15] Y. Wang, Q. Chen, and T. W. Kuan, "An application of machine vision for autonomous robots – Toy identification and classification," in *2022 10th International Conference on Orange Technology (ICOT)*, pp. 1–4, 2022, doi: 10.1109/ICOT56925.2022.10008139.
- [16] A. Findikçi, M. Y. Erten, and H. Aydılek, "Melan-YOLO: a dual-strategy framework with custom architectures for accurate and efficient underwater object detection," *Scientific Reports*, 2026, doi: 10.1038/s41598-026-55023-9.
- [17] B. Song et al., "YOLO-SAD: Enhancing Small Arthropod Target Detection for Autonomous Port Inspection Robots," *IEEE Sensors Journal*, vol. 26, no. 3, pp. 4406–4421, 2026, doi: 10.1109/JSEN.2025.3643408.
- [18] S. Zhou, H. Zhou, and L. Qian, "A multi-scale small object detection algorithm SMA-YOLO for UAV remote sensing images," *Scientific Reports*, vol. 15, no. 1, pp. 1–15, 2025, doi: 10.1038/s41598-025-92344-7.
- [19] H. Jati, N. A. Ilyasa, and D. D. Dominic, "Enhancing Humanoid Robot Soccer Ball Tracking, Goal Alignment, and Robot Avoidance Using YOLO-NAS," *Journal of Robotics and Control*, vol. 5, no. 3, pp. 829–838, 2024, doi: 10.18196/jrc.v5i3.21839.
- [20] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE CVPR*, pp. 779–788, 2016.
- [21] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proc. IEEE CVPR*, pp. 7263–7271, 2017.
- [22] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv:1804.02767*, 2018.
- [23] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv:2004.10934*, 2020.
- [24] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," *arXiv:2207.02696*, 2022.
- [25] B. J. Erickson and F. Kitamura, "Magician's corner: 9. performance metrics for machine learning models," *Radiol. Artif. Intell.*, vol. 3, no. 3, pp. 1–7, 2021, doi: 10.1148/ryai.2021200126.